

BOOTS DIGITAL HEALTH · CLINICAL OPERATIONS

BLeaf Clinical Workforce Management Tool

Product Requirements Document

Document Type	Product Requirements Document
Version	1.0 — Prototype → Production
Date	June 2026
Technology Stack	TypeScript · Node.js · Python
Audience	Development team, QA, Product
Status	Draft for review

Generated from a working prototype. All code snippets are executable TypeScript, Python, or Node.js. Screenshots and wireframes are available in the live demo linked in each epic.

BLeaf Clinical Workforce Management Tool — Product Requirements

Version: 1.0 (prototype → production)

Prepared from: Working prototype (bleaf-demo)

Technology stack: TypeScript · React 18 · Node.js · Python

Service: Boots Digital Health (BLeaf online prescribing platform)

1. System Overview

BLeaf is a clinical operations tool used by the medical management team at Boots Digital Health. The tool manages how online prescribers (pharmacists, nurses, GPs, specialists) are deployed each day to handle patient orders and messages across seven clinical service categories.

Primary users:

- **Medical Manager (MM)** — oversees day-to-day clinical operations
- **Resource Manager (RM)** — allocates staff to queues
- **Duty Pharmacist / Senior Prescriber** — monitors live queue health

Working hours: 08:00–20:00 (12-hour prescribing day)

Core workflow each day:

1. MM/RM opens dashboard at start of shift
 2. Reviews current BODCON level (operational risk rating)
 3. Auto-allocates (or manually drags) prescribers into service category tiles
 4. Monitors performance flags throughout the day
 5. Applies breaks on schedule; moves prescribers to non-prescribing slots for admin/training
 6. Reviews projected demand vs capacity before busy periods
-

2. Domain Model & Data Types

2.1 TypeScript Interfaces (canonical source of truth)

```

// Prescriber roles – determines which services a prescriber can handle
type PrescriberRole = 'pharmacist' | 'nurse' | 'gp' | 'specialist';

// Prescriber lifecycle states
type PrescriberStatus =
  | 'online'           // Available in pool, not yet allocated
  | 'allocated'       // Assigned to a service category tile
  | 'scheduled'       // On rota but not yet started shift
  | 'offline'         // Not working today
  | 'on-break'        // Temporary break (auto-reverts when break ends)
  | 'in-appointment'  // In a scheduled patient appointment
  | 'non-prescribing'; // Admin, training, meeting, lunch

type OrderStatus = 'pending' | 'allocated' | 'in-progress' | 'complete' | 'escalated';
type Urgency = 'routine' | 'urgent' | 'critical';

interface Prescriber {
  id: string;
  name: string;
  initials: string;           // 2-3 characters, used in avatar
  role: PrescriberRole;
  serviceIds: string[];      // Services this prescriber is qualified for
  status: PrescriberStatus;
  avatar?: string;           // Optional profile image URL
  allocatedCategoryId?: string; // Set when status === 'allocated'
}

interface Order {
  id: string;
  serviceId: string;
  patientRef: string;        // Anonymised patient reference
  urgency: Urgency;
  ageHours: number;          // Hours since order was placed
  value: number;              // Order value in GBP
  prescriberId?: string;     // Set when allocated/in-progress
  status: OrderStatus;
  createdAt: string;         // ISO 8601
  priorityScore?: number;    // Computed – see §8
}

interface ServiceCategory {
  id: string;
  name: string;
  color: string;              // Hex, used for tile theming
  icon: string;               // Emoji or unicode symbol
  serviceIds: string[];      // Child service IDs
}

interface Service {
  id: string;
  name: string;
  categoryId: string;
  slaHours: number;           // Target completion time
  requiredRoles: PrescriberRole[]; // Roles permitted to handle this service
}

interface ServiceCapacityConfig {
  categoryId: string;
}

```

```

    orderAHTMins: number;        // Average handling time per order (minutes)
    messageAHTMins: number;      // Average handling time per patient message (minutes)
}

interface AllocationRule {
    id: string;
    name: string;
    description: string;
    enabled: boolean;
    weight: number;
    field: 'urgency' | 'ageHours' | 'value' | 'slaHours' | 'role';
    operator: 'gt' | 'lt' | 'eq' | 'gte' | 'lte';
    value: number | string;
    action: 'boost' | 'deprioritise' | 'escalate';
    actionValue: number;
}

interface SLAConfig {
    categoryId: string;
    targetHours: number;
    warningThresholdPct: number; // Default 60% elapsed → amber
    criticalThresholdPct: number; // Default 85% elapsed → red
}

interface DayAllocation {
    categoryId: string;
    prescriberIds: string[];
}

interface Appointment {
    id: string;
    patientRef: string;
    clinicTypeId: string;
    prescriberId: string;
    startTime: string;          // "HH:MM" 24-hour
    durationMins: number;
    notes?: string;
    status: 'scheduled' | 'in-progress' | 'complete' | 'cancelled';
}

interface ClinicType {
    id: string;
    name: string;
    color: string;
    defaultDurationMins: number;
    requiredRoles: PrescriberRole[];
}

interface BreakGroup {
    id: string;
    name: string;
    startTime: string;          // "HH:MM"
    endTime: string;            // "HH:MM"
    prescriberIds: string[];
    enabled: boolean;
    color: string;
}

```

```

interface PatientMessage {
  id: string;
  serviceId?: string;
  categoryId?: string;
  patientRef: string;
  urgency: Urgency;
  ageHours: number;
  status: 'pending' | 'allocated' | 'complete';
  isGeneral: boolean;
  prescriberId?: string;
  priorityScore?: number;
}

interface NonPrescribingSlot {
  prescriberId: string;
  reason: 'admin' | 'training' | 'meeting' | 'lunch' | 'other';
  note?: string;
}

interface PrescriberActivityEvent {
  id: string;
  prescriberId: string;
  type: 'order' | 'message';
  timestamp: string; // ISO 8601
}

interface PerformanceMonitorConfig {
  slowRateThresholdPct: number; // % below team average before flagging (default 20)
  watchHours: number; // Hours below threshold → Watch flag (default 1)
  actionHours: number; // Hours below threshold → Take Action flag (default 2)
  idleMinutes: number; // Minutes with no activity → Idle flag (default 20)
}

```

2.2 Service Categories & Default AHT Values

Category	Order AHT	Message AHT	SLA Target	Services
Women's Health	10 min	5 min	24 h	Contraception, HRT, Menopause, Cystitis, Period Delay... (10 services)
Men's Health	10 min	5 min	48 h	ED, Hair Loss, Premature Ejaculation, Testosterone... (8 services)
Weight Management	12 min	6 min	24 h	GLP-1, Coaching, BMI, Metabolic Health... (6 services)
Sexual Health	8 min	4 min	12 h	STI, PrEP, HIV, Chlamydia, Gonorrhoea... (8 services)
Dermatology	9 min	4 min	48 h	Acne, Rosacea, Eczema, Psoriasis, Cold Sores... (6 services)
Mental Health	15 min	8 min	72 h	Anxiety, Depression (SSRI), Sleep, ADHD... (6 services)
General Health	8 min	4 min	24 h	Hay Fever, Migraine, Smoking Cessation, Travel Health (4 services)

2.3 Prescriber Role → Service Eligibility

```
Pharmacist → can handle most Women's, Men's, Sexual Health, Dermatology, General Health
Nurse      → broader scope; can handle Weight Management, Mental Health
GP         → all categories except Specialist-only services
Specialist → required for HRT/Menopause/complex cases, GLP-1, ADHD, HIV, Fertility
```

3. Epic 1 — BODCON Operational Status

BODCON (Business Operations CONdition) is a 5-level operational risk rating visible at all times as a compact 46px banner at the top of the dashboard. It mirrors escalation frameworks used in emergency services.

3.1 BODCON Level Definitions

Level	Label	Description	Trigger
1	BODCON 1	Critical	≥4 red guardrails, or ≥3 red + ≥3 amber
2	BODCON 2	Severe	≥3 red, or ≥2 red + ≥3 amber, or ≥1 red + ≥6 amber
3	BODCON 3	Elevated	≥2 red, or ≥6 amber, or ≥1 red + ≥3 amber
4	BODCON 4	Monitoring	≥1 red, or ≥3 amber
5	BODCON 5	Normal	≤2 amber, or all green

3.2 RAG Status per Category (Guardrail)

Each service category is independently RAG-rated based on its **current pending queue** only:

```
gap = requiredMins - availableMins

requiredMins = (pendingOrders × orderAHTMins) + (pendingMessages × messageAHTMins)
availableMins = allocatedPrescriberCount × 480    (8-hour effective prescribing day)

RAG:
gap ≤ 0      → GREEN  (capacity sufficient)
0 < gap ≤ 180 → AMBER  (up to 3 prescriber-hours behind)
gap > 180    → RED    (more than 3 prescriber-hours behind)
```

Design decision: BODCON is driven by current queue only, not projected demand. Projected demand drives a separate, non-alarming advisory warning (amber ▲ text in the banner). This prevents "phantom" escalations early in the day when queues are naturally building.

3.3 Projected Demand Warning

A separate, non-blocking advisory appears in the banner when projected remaining orders would worsen a category's RAG:

```
// Python equivalent for backend calculation
def compute_projected_rag(
    current_req_mins: float,
    avail_mins: float,
    expected_today: int,
    hours_elapsed: float,
    work_hours: float = 12.0,
    order_aht_mins: float = 10.0,
) → str:
    hours_remaining = max(0, work_hours - hours_elapsed)
    fraction_remaining = hours_remaining / work_hours
    projected_remaining = round(expected_today * fraction_remaining)
    projected_req = current_req_mins + (projected_remaining * order_aht_mins)
    gap = projected_req - avail_mins
    if gap ≤ 0:
        return "green"
    if gap ≤ 180:
        return "amber"
    return "red"
```

3.4 User Stories

US-BOD-01

As a Medical Manager, I want to see the current BODCON level at a glance at the top of the dashboard so that I immediately know the operational risk without having to read a dashboard in detail.

Acceptance Criteria:

- [] BODCON badge is visible on every screen within the tool (not just the home dashboard)
- [] Badge shows the numeric level (1–5) and a short description ("Normal operations", "Elevated", etc.)
- [] Background and border colour reflects severity: red (1), orange (2), amber (3/4), neutral (5)
- [] Banner is exactly 46px tall — compact enough not to dominate the screen
- [] Banner refreshes automatically when order/message/allocation state changes (no page reload required)

Suggested tests:

```
// Unit test: BODCON level computation
import { computeBodconLevel } from './bodconConfig';

describe('computeBodconLevel', () => {
  it('returns 1 when 4 or more guardrails are red', () => {
    expect(computeBodconLevel(4, 0)).toBe(1);
    expect(computeBodconLevel(5, 0)).toBe(1);
  });
  it('returns 1 when 3 red and 3 amber', () => {
    expect(computeBodconLevel(3, 3)).toBe(1);
  });
  it('returns 2 for 3 red and 2 amber', () => {
    expect(computeBodconLevel(3, 2)).toBe(2);
  });
  it('returns 5 for all green', () => {
    expect(computeBodconLevel(0, 0)).toBe(5);
  });
  it('returns 5 for 2 amber', () => {
    expect(computeBodconLevel(0, 2)).toBe(5);
  });
  it('returns 4 for 3 amber', () => {
    expect(computeBodconLevel(0, 3)).toBe(4);
  });
});

// Unit test: RAG status
describe('ragStatus', () => {
  it('returns green when capacity exceeds demand', () => {
    expect(ragStatus(500, 400)).toBe('green');
  });
  it('returns amber for gap of 1-180 mins', () => {
    expect(ragStatus(400, 500)).toBe('amber'); // gap = 100
    expect(ragStatus(320, 500)).toBe('amber'); // gap = 180
  });
  it('returns red for gap > 180 mins', () => {
    expect(ragStatus(300, 500)).toBe('red'); // gap = 200
  });
});
```

US-BOD-02

As a Medical Manager, I want to see how many guardrails are red, amber, and green at a glance so that I understand which service areas are under pressure.

Acceptance Criteria:

- [] Three RAG dot counters are always shown (N, N, N) even when count is 0
- [] Counts update in real-time as orders arrive or prescribers are allocated
- [] Clicking "Details >" opens a modal with per-category breakdown including: pending orders, pending messages, allocated prescriber count, current RAG, and projected RAG badge if worse

US-BOD-03

As a Medical Manager, I want a projected pressure warning to appear when expected remaining order volume would breach capacity so that I can plan staffing proactively.

Acceptance Criteria:

- [] Warning only appears when at least one category's projected RAG is worse than its current RAG
- [] Warning text truncates with ... if too long for the banner — full text shows on hover (title attribute)
- [] "Details >" button is always visible and never obscured by the warning text
- [] Warning is amber, not red — it must not be confused with a current-state alert
- [] The historical order volume data used for projection is stored by service category and day-of-week (Sunday=0 to Saturday=6)

```
// Expected daily volumes by category and day-of-week
// [Sun, Mon, Tue, Wed, Thu, Fri, Sat]
const DAILY_ORDER_CONFIG = [
  { categoryId: 'womens-health',    expectedOrders: [1968, 2150, 1767, 1609, 1471, 1537, 1685] },
  { categoryId: 'weight-management', expectedOrders: [1801, 2035, 1886, 1890, 1896, 1967, 1535] },
  { categoryId: 'mens-health',      expectedOrders: [209, 311, 275, 259, 278, 263, 218] },
  { categoryId: 'sexual-health',    expectedOrders: [340, 420, 380, 350, 320, 380, 410] },
  { categoryId: 'dermatology',      expectedOrders: [311, 307, 318, 344, 343, 304, 287] },
  { categoryId: 'mental-health',    expectedOrders: [120, 180, 165, 155, 150, 160, 130] },
  { categoryId: 'general-health',   expectedOrders: [187, 280, 184, 185, 228, 194, 169] },
];
```

4. Epic 2 — Service Capacity Tiles

The main grid shows one tile per service category. Tiles are the central unit of workforce allocation — prescribers are dragged on to tiles and the tile's RAG indicator shows live capacity status.

4.1 Tile Layout

[Icon]

Category Name

N services

[badges]

N orders

N msgs

[Avatar] Dr Name ×

[Avatar] Dr Name ×

[Avatar] Dr Name ×

[· · · + drop another · · ·]

▼ Show all 8

← 3px RAG-coloured bar (top edge)

← Header

← Prescriber rows (scrollable)

← Max 224px height (~5 rows)

← Expand toggle (when >4 allocated)

4.2 User Stories

US-TIL-01

As a Resource Manager, I want to drag prescribers from the right-hand pool panel onto a service tile so that I can allocate them to handle that category's orders.

Acceptance Criteria:

- [] Prescriber card in pool panel is draggable
- [] Valid drop targets (tiles where the prescriber is qualified) show a coloured dashed border on hover
- [] Invalid drop targets (categories with no matching service IDs) dim to 50% opacity and reject the drop
- [] On successful drop, prescriber moves from pool to the tile and their status changes from **online** → **allocated**
- [] Prescriber can be dragged from one tile to another (moving, not duplicating)
- [] Prescriber can be dragged back to the pool panel to deallocate (status reverts to **online**)

Skill matching rule:

```
// A prescriber can work in a category if they share at least one service ID
const canWork = category.serviceIds.some(sId => prescriber.serviceIds.includes(sId));
```

US-TIL-02

As a Resource Manager, I want tile prescriber lists to scroll internally when there are many allocations so that the tile height stays fixed and the drag-drop area remains usable regardless of team size.

Acceptance Criteria:

- [] Prescriber list within each tile has a **maxHeight** of 224px (~5 rows) and **overflowY: auto**
- [] A "▼ Show all N" expand button appears when more than 4 prescribers are allocated
- [] Clicking the toggle expands to show all prescribers; label changes to "▲ Show less"
- [] The outer tile size does not force the page to scroll when many prescribers are allocated
- [] Drop zone ("+ drop another") remains visible at the bottom of the scrollable list

US-TIL-03

As a Resource Manager, I want to right-click a prescriber in a tile to move them to another tile or return them to the pool without drag-and-drop so that allocation is accessible and efficient.

Acceptance Criteria:

- [] Right-clicking a prescriber in a tile opens a context menu at cursor position
- [] Context menu shows: prescriber name header; "Move to" section listing only eligible categories (not the current one); "↩ Return to pool" action
- [] Only categories where the prescriber has a matching service ID are shown as move targets
- [] Clicking a move target removes the prescriber from the current tile and adds them to the target tile
- [] Clicking outside the menu dismisses it without making changes
- [] Context menu is fixed-position (`position: fixed`) so it is never clipped by scrollable containers

```
// Context menu target computation
const targets = SERVICE_CATEGORIES.filter(cat =>
  cat.id !== currentCategoryId &&
  cat.serviceIds.some(sId => prescriber.serviceIds.includes(sId))
);
```

US-TIL-04

As a Resource Manager, I want the tile to show a red, amber, or green indicator bar so that I can see which categories are under capacity pressure at a glance.

Acceptance Criteria:

- [] 3px bar at the top edge of each tile is coloured: green (#2E7D32), amber (#D97706), or red (#DC2626)
- [] RAG status updates in real-time when orders arrive or prescribers are allocated/deallocated
- [] Tile header badges show: pending order count (colour by severity) and pending message count

5. Epic 3 — Prescriber Workforce Management

5.1 User Stories

US-POOL-01

As a Resource Manager, I want to search for a prescriber by name in the right-hand panel so that I can quickly locate prescribers who may be hidden in a long scrollable tile.

Acceptance Criteria:

- [] Search input at top of pool panel filters all 55+ prescribers in real-time as the user types
- [] Search results show: prescriber avatar, name, role, and the category tile they are currently

allocated to (in the category's colour)

- [] If a prescriber is not allocated, their current status is shown instead (online, offline, on-break, etc.)
- [] Search is case-insensitive and matches on partial name
- [] A × button clears the search and returns to the standard pool view
- [] Draggable prescribers (those with status `online`) remain draggable from search results

US-POOL-02

As a Resource Manager, I want the pool panel to group prescribers into Available Pool, Scheduled, and Offline sections so that I can understand who is available to allocate.

Acceptance Criteria:

- [] "Available Pool" section shows prescribers with status `online` (draggable, shown in full colour)
- [] "Scheduled" section shows prescribers with status `scheduled` (muted, not draggable, shows "Sched" badge)
- [] "Offline" section shows prescribers with status `offline` (greyscale, shows "Log in" button to set `online`)
- [] When all online prescribers are allocated, shows "All online prescribers allocated" message

US-POOL-03

As a Resource Manager, I want to click `Auto-allocate` to distribute all available prescribers proportionally across service categories so that I don't have to manually allocate 50+ people every morning.

Acceptance Criteria:

- [] All prescribers with status `online` or `scheduled` are allocated; nobody is left in the pool after auto-allocation (unless no eligible category exists)
- [] Prescribers are distributed proportionally to each category's `requiredMins` (AHT × workload)
- [] Categories with zero workload receive zero prescribers
- [] Role/skill constraints are respected — a prescriber is only allocated to a category where `canWork` is true
- [] After allocation, order `priorityScore` values are recomputed for all orders

Auto-allocate algorithm:

```

// Step 1: Compute required minutes per category
const workload = categories.map(cat => ({
  categoryId: cat.id,
  reqMins: pendingOrders[cat.id] * aht.orderMins + pendingMsgs[cat.id] * aht.msgMins,
}));

// Step 2: Proportional targets (each prescriber is assigned exactly once)
const n = available.length;
const totalReq = workload.reduce((s, w) => s + w.reqMins, 0);
let targets = workload.map(w => ({
  categoryId: w.categoryId,
  target: w.reqMins > 0 ? Math.max(1, Math.round(n * w.reqMins / totalReq)) : 0,
}));

// Step 3: Normalise so sum(targets) ≡ n
// Trim over-count from smallest categories first, then add remainder to busiest

// Step 4: Assign eligible prescribers (skill-filtered) to each category by target
// Step 5: Assign remaining unallocated prescribers to best-fit category by workload

```



```

# Python equivalent (for backend validation or batch allocation)
import math
from typing import List, Dict

def auto_allocate(
    prescribers: List[dict],
    categories: List[dict],
    workload: Dict[str, float], # categoryId → requiredMins
) → Dict[str, str]: # prescriberId → categoryId
    """Returns a mapping of prescriberId → categoryId."""
    available = [p for p in prescribers if p['status'] in ('online', 'scheduled')]
    n = len(available)
    total_req = sum(workload.values())
    if total_req == 0 or n == 0:
        return {}

    # Proportional targets
    targets = {}
    for cat in categories:
        req = workload.get(cat['id'], 0)
        targets[cat['id']] = max(1, round(n * req / total_req)) if req > 0 else 0

    # Normalise
    total = sum(targets.values())
    sorted_cats = sorted(targets.keys(), key=lambda k: targets[k])
    while total > n:
        for cid in sorted_cats:
            if targets[cid] > 1:
                targets[cid] -= 1
                total -= 1
                break
    if total < n:
        busiest = max(targets, key=lambda k: targets[k])
        targets[busiest] += n - total

    # Assign
    result = {}
    assigned = set()
    for cid in sorted(targets, key=lambda k: -targets[k]):
        cat = next(c for c in categories if c['id'] == cid)
        eligible = [
            p for p in available
            if p['id'] not in assigned
            and any(sid in p['serviceIds'] for sid in cat['serviceIds'])
        ]
        for p in eligible[:targets[cid]]:
            result[p['id']] = cid
            assigned.add(p['id'])

    return result

```

US-POOL-04

As a Resource Manager, I want to clear all allocations with one click so that I can reset and re-allocate if the team changes at the start of a shift.

Acceptance Criteria:

- [] "Clear allocations" button returns all allocated prescribers to **online** status
- [] All **DayAllocation** records are reset to empty **prescriberIds** arrays
- [] Action does not affect non-prescribing slots, breaks, or appointments

6. Epic 4 — Performance Monitoring

The Performance Monitor is a 220px left-panel that flags prescribers who are processing orders/messages below the team average rate.

6.1 Flag Levels

Flag	Colour	Trigger	Meaning
Take Action	Red	Below threshold for $\geq$ actionHours	Speak to the prescriber — something may be wrong
⏸ Idle	Purple	No activity for $\geq$ idleMinutes	May have forgotten to move to non-prescribing
👁 Watch	Amber	Below threshold for $\geq$ watchHours	Keep an eye on this prescriber

6.2 Computation Logic

```
// Default thresholds (all configurable)
const defaultConfig: PerformanceMonitorConfig = {
  slowRateThresholdPct: 20, // Flag if >20% below team average
  watchHours: 1,           // Watch flag after 1 hour below threshold
  actionHours: 2,          // Take Action flag after 2 hours below threshold
  idleMinutes: 20,         // Idle flag after 20 minutes of no activity
};

// Per prescriber, computed over rolling time windows:
const watchStart = now - (watchHours * 3_600_000);
const actionStart = now - (actionHours * 3_600_000);

const watchRate = eventsInWatchWindow / watchHours; // events per hour
const actionRate = eventsInActionWindow / actionHours; // events per hour

// Team average = mean of watch-window rates, EXCLUDING zeros
// (so freshly allocated prescribers don't pull down the baseline)
const avgRate = mean(stats.filter(s => s.watchRate > 0).map(s => s.watchRate));

const threshold = avgRate * (1 - slowRateThresholdPct / 100);
const watchSlow = watchRate < threshold;
const actionSlow = actionRate < threshold;

// Flag assignment:
if (isIdle && !(watchSlow && actionSlow)): flag = 'idle'
elif watchSlow && actionSlow:             flag = 'action'
elif watchSlow:                           flag = 'watch'
```

```

# Python equivalent for backend performance analysis
from datetime import datetime, timedelta
from statistics import mean
from typing import Optional

def compute_performance_flags(
    prescribers: list[dict],
    activity_events: list[dict],
    config: dict,
    now: Optional[datetime] = None,
) → list[dict]:
    if now is None:
        now = datetime.utcnow()

    watch_ms = timedelta(hours=config['watchHours'])
    action_ms = timedelta(hours=config['actionHours'])
    idle_ms = timedelta(minutes=config['idleMinutes'])
    threshold_factor = 1 - config['slowRateThresholdPct'] / 100

    allocated = [p for p in prescribers if p['status'] == 'allocated']
    if not allocated:
        return []

    stats = []
    for p in allocated:
        events = [e for e in activity_events if e['prescriberId'] == p['id']]
        timestamps = [datetime.fromisoformat(e['timestamp'].replace('Z', '+00:00')) for e in events]

        watch_count = sum(1 for t in timestamps if t ≥ now - watch_ms)
        action_count = sum(1 for t in timestamps if t ≥ now - action_ms)
        last_ts = max(timestamps, default=None)

        stats.append({
            'prescriber': p,
            'watchRate': watch_count / config['watchHours'],
            'actionRate': action_count / config['actionHours'],
            'lastTs': last_ts,
        })

    rates_with_data = [s['watchRate'] for s in stats if s['watchRate'] > 0]
    avg_rate = mean(rates_with_data) if rates_with_data else 0
    if avg_rate == 0:
        return []

    threshold = avg_rate * threshold_factor
    flags = []

    for s in stats:
        is_idle = s['lastTs'] is not None and (now - s['lastTs']) > idle_ms
        watch_slow = s['watchRate'] < threshold
        action_slow = s['actionRate'] < threshold

        if is_idle and not (watch_slow and action_slow):
            level = 'idle'
        elif watch_slow and action_slow:
            level = 'action'
        elif watch_slow:
            level = 'watch'

```

```

else:
    continue

flags.append({
    'prescriberId': s['prescriber']['id'],
    'name': s['prescriber']['name'],
    'level': level,
    'recentRate': round(s['watchRate'], 1),
    'avgRate': round(avg_rate, 1),
    'lastActivityMinsAgo': (
        int((now - s['lastTs']).total_seconds() / 60)
        if s['lastTs'] else None
    ),
})

return sorted(flags, key=lambda f: {'action': 0, 'idle': 1, 'watch': 2}[f['level']])

```

6.3 User Stories

US-PERF-01

As a Medical Manager, I want the Performance Monitor panel to automatically flag prescribers who are processing orders significantly below the team average so that I can identify if someone needs support.

Acceptance Criteria:

- [] Panel refreshes every 30 seconds without user action
- [] Flags only appear when there is a baseline average (at least one prescriber with activity data)
- [] Each flag card shows: prescriber avatar, name, role, rate vs team average, and time below threshold
- [] Sections are ordered: Take Action first, then Idle, then Watch
- [] Empty state shows "All prescribers on track" when no flags exist

US-PERF-02

As a Medical Manager, I want to configure the performance thresholds so that the flags are appropriate for our service and team.

Acceptance Criteria:

- [] ⚙ gear icon opens a configuration modal
- [] Four configurable fields: slow rate threshold (%), watch hours, take-action hours, idle minutes
- [] All inputs are numeric with sensible min/max: threshold 5–80%, watch 0.25–4h, action 0.5–8h, idle 5–120min
- [] Changes take effect immediately on save without page reload

Suggested tests:

```
describe('PerformanceMonitor flag computation', () => {
  const baseConfig = {
    slowRateThresholdPct: 20,
    watchHours: 1,
    actionHours: 2,
    idleMinutes: 20,
  };

  it('flags a prescriber as "watch" when below threshold for watch window', () => {
    const now = Date.now();
    // Team: prescriber A (6/h), prescriber B (1/h) – B is 83% below average
    const events = [
      ...Array.from({ length: 6 }, (_, i) => ({
        id: `a${i}`, prescriberId: 'p-a', type: 'order',
        timestamp: new Date(now - i * 600_000).toISOString(), // every 10 min
      })),
      {
        id: 'b1', prescriberId: 'p-b', type: 'order',
        timestamp: new Date(now - 3_000_000).toISOString(), // 50 min ago
      },
    ];
    // B's watch-rate ≈ 1/h, team avg ≈ 3.5/h, threshold = 2.8/h → B should be Watch
    const flags = computePerformanceFlags(prescribers, events, baseConfig, new Date(now));
    expect(flags.find(f => f.prescriberId === 'p-b')?.level).toBe('watch');
  });

  it('escalates to "action" when below threshold for action window', () => {
    // B has been slow for 2+ hours
    // ... (setup events 3 hours ago for B, recent events for A)
  });

  it('flags as "idle" when last event was >20 minutes ago', () => {
    // B had one event 25 minutes ago, A is active
  });
});
```

7. Epic 5 — Appointment Diary

The Appointment Diary provides a calendar grid for the prescribing day (08:00–20:00) with columns per clinic type (not per prescriber). This allows the operations team to see all scheduled patient appointments and manage them.

7.1 Grid Structure

- **Time column:** 30-minute slot rows, 08:00–20:00 (24 slots)
- **Clinic type columns:** One column per configured clinic type (e.g. Video Consultation, Async Review, Phone Clinic)
- **Cell height:** 44px per 30-min slot
- **Column width:** 180px per clinic type

7.2 User Stories

US-DIARY-01

As a Medical Manager, I want to see all today's appointments in a time-grid view so that I can understand prescriber capacity committed to appointments vs available for orders.

Acceptance Criteria:

- [] Grid renders all appointments for the current day
- [] Appointments are displayed in the correct clinic type column and time slot
- [] Multi-slot appointments (>30 min) visually span the correct number of rows (via continuation markers)
- [] Break groups appear as coloured bands across all columns during the break period
- [] Grid is horizontally scrollable if there are many clinic types

US-DIARY-02

As a Resource Manager, I want to click a time slot to add a new appointment so that I can schedule patient consultations.

Acceptance Criteria:

- [] Clicking an empty slot opens the New Appointment modal, pre-filled with the clinic type and time
- [] Modal requires: patient reference (PT-XXXX format), prescriber, start time, duration
- [] Prescriber dropdown only shows prescribers eligible for the selected clinic type (matching `requiredRoles`)
- [] Duration defaults to the clinic type's `defaultDurationMins` (15–480 min, in 15-min steps)
- [] Saved appointment appears immediately in the grid

US-DIARY-03

As a Resource Manager, I want to click an existing appointment to edit or cancel it.

Acceptance Criteria:

- [] Clicking a single appointment in a slot opens the Edit Appointment modal
- [] Edit modal allows changing: patient ref, prescriber, start time, duration, status, notes
- [] "Cancel appt" sets status to `cancelled` (removes from view, retains in data for audit)
- [] "Delete" removes the appointment entirely
- [] If multiple appointments start at the same slot, a count badge appears ("3 · click to expand")
- [] Clicking the count badge opens an expanded popover listing all appointments in that slot

As a Resource Manager, I want to create and configure clinic types with custom colours and role requirements so that the diary reflects our actual appointment types.

Acceptance Criteria:

- [] "+ Clinic Type" button opens the New Clinic Type modal
- [] Required fields: name, colour (selected from 8 preset colours), default duration
- [] Optional: required roles (multi-select: Pharmacist, Nurse, GP, Specialist)
- [] Existing clinic types are shown as clickable pills in the toolbar for editing
- [] Clinic type colour appears as a 3px top border on the diary column header

Suggested tests:

```
describe('AppointmentDiary', () => {
  it('computes correct slot index from time string', () => {
    expect(timeToSlot('08:00')).toBe(0);
    expect(timeToSlot('08:30')).toBe(1);
    expect(timeToSlot('12:00')).toBe(8);
    expect(timeToSlot('20:00')).toBe(24);
  });

  it('returns correct slot time from index', () => {
    expect(slotToTime(0)).toBe('08:00');
    expect(slotToTime(8)).toBe('12:00');
  });

  it('computes slots occupied by a 60-min appointment at 09:00', () => {
    expect(getAppointmentSlots('09:00', 60)).toEqual([2, 3]);
  });

  it('filters eligible prescribers by clinic type required roles', () => {
    const clinicType = { requiredRoles: ['nurse', 'gp'] };
    const eligible = prescribers.filter(p =>
      clinicType.requiredRoles.includes(p.role) && p.status !== 'offline'
    );
    expect(eligible.every(p => ['nurse', 'gp'].includes(p.role))).toBe(true);
  });
});
```

8. Epic 6 — Order Priority & Allocation Rules

8.1 Priority Score Calculation

Each order has a numeric **priorityScore** computed from base urgency, SLA proximity, and configurable allocation rules. Higher scores = higher priority.


```
function computePriorityScore(order: Order, rules: AllocationRule[], slas: SLAConfig[]): number {
  const service = SERVICES.find(s => s.id === order.serviceId);
  const sla = slas.find(s => service && s.categoryId === service.categoryId);
  let score = 0;

  // Base urgency score
  const urgencyBase = order.urgency === 'critical' ? 80 : order.urgency === 'urgent' ? 40 : 0;
  score += urgencyBase;

  // SLA proximity
  if (sla) {
    const pctElapsed = (order.ageHours / sla.targetHours) * 100;
    if (pctElapsed ≥ sla.criticalThresholdPct) score += 60; // >85% elapsed
    else if (pctElapsed ≥ sla.warningThresholdPct) score += 30; // >60% elapsed
  }

  // Configurable rules
  for (const rule of rules.filter(r => r.enabled)) {
    const match = evaluateRule(rule, order);
    if (match) {
      if (rule.action === 'boost') score += rule.actionValue;
      else if (rule.action === 'deprioritise') score -= rule.actionValue;
      else if (rule.action === 'escalate') score += 200; // Forces to top of queue
    }
  }

  return Math.max(0, score);
}
```

8.2 Default Allocation Rules

ID	Name	Condition	Action	Value	Enabled
rule-01	Critical Urgency Boost	urgency = critical	boost	+100	
rule-02	Urgent Order Boost	urgency = urgent	boost	+50	
rule-03	SLA Breach Risk	ageHours > 20	boost	+60	
rule-04	High Value Order	value > £60	boost	+20	
rule-05	Stale Order Flag	ageHours > 36	escalate	—	
rule-06	Routine Deprioritise	urgency = routine	deprioritise	-10	

8.3 Default SLA Targets

Category	Target	Warning at	Critical at
Women's Health	24h	60% (14.4h)	85% (20.4h)
Men's Health	48h	60% (28.8h)	85% (40.8h)
Weight Management	24h	60% (14.4h)	85% (20.4h)
Sexual Health	12h	60% (7.2h)	85% (10.2h)
Dermatology	48h	60% (28.8h)	85% (40.8h)
Mental Health	72h	60% (43.2h)	85% (61.2h)
General Health	24h	60% (14.4h)	85% (20.4h)

8.4 User Stories

US-RULE-01

As a Medical Manager, I want to configure allocation rules so that the system prioritises orders in line with our clinical policies.

Acceptance Criteria:

- [] Rules list shows all rules with: name, description, condition, action, enabled toggle
- [] Rules can be enabled/disabled without deleting them
- [] Rule fields: `field` (urgency/ageHours/value), `operator` (gt/lt/eq/gte/lte), `value`, `action` (boost/deprioritise/escalate), `actionValue`
- [] Changes to rules trigger re-scoring of all pending orders
- [] `escalate` action adds 200 to the score (guarantees top of queue regardless of other rules)

US-RULE-02

As a Medical Manager, I want to configure SLA targets per category so that the priority scoring reflects our service commitments.

Acceptance Criteria:

- [] Each category has configurable `targetHours`, `warningThresholdPct`, `criticalThresholdPct`
- [] Warning at 60% elapsed and critical at 85% are defaults but must be overridable
- [] SLA configuration changes take effect immediately on existing orders

9. Epic 7 — Break Management

9.1 User Stories

US-BRK-01

As a Resource Manager, I want to define break groups (e.g. "Morning break 10:00–10:15") so that I can schedule when batches of prescribers go on break.

Acceptance Criteria:

- [] Break groups have: name, start time, end time, colour, list of prescribers, enabled flag
- [] Groups are configurable via the Diary view's toolbar
- [] A break group can contain any subset of prescribers
- [] Multiple groups can overlap in time (different cohorts taking staggered breaks)

US-BRK-02

As a Resource Manager, I want to click " Apply Breaks" to automatically move prescribers on break now from their allocated tiles.

Acceptance Criteria:

- [] "Apply Breaks" checks the current clock time against all enabled break groups
- [] If the current time falls within a break group's window, all prescribers in that group are moved from `allocated` → `on-break`
- [] Prescribers on break are removed from their category tile's `prescriberIds`
- [] If no break groups are currently active, the action has no effect
- [] Break status is transient — prescribers can be re-allocated manually after break

```
// Break application logic
function applyBreaks(state: State): State {
  const now = new Date();
  const nowMins = now.getHours() * 60 + now.getMinutes();

  const activeBreaks = state.breakGroups.filter(bg => {
    if (!bg.enabled) return false;
    const start = timeToMins(bg.startTime);
    const end = timeToMins(bg.endTime);
    return nowMins ≥ start && nowMins < end;
  });

  const prescriberIdsToBreak = new Set(
    activeBreaks.flatMap(bg => bg.prescriberIds)
  );

  // Only moves prescribers who are currently allocated
  // (doesn't affect non-prescribing or appointment statuses)
  return {
    ...state,
    allocations: state.allocations.map(a => ({
      ...a,
      prescriberIds: a.prescriberIds.filter(id => !prescriberIdsToBreak.has(id)),
    })),
    prescribers: state.prescribers.map(p =>
      prescriberIdsToBreak.has(p.id) && p.status === 'allocated'
        ? { ...p, status: 'on-break', allocatedCategoryId: undefined }
        : p
    ),
  };
}
}
```

Suggested tests:

```
describe('applyBreaks', () => {
  it('moves allocated prescribers to on-break during active break window', () => {
    // Mock current time to 10:05
    // Setup break group 10:00-10:15 containing prescriber A
    // A should become 'on-break'
  });

  it('does not affect prescribers in non-prescribing status', () => {
    // Prescriber B is 'non-prescribing' and in the break group
    // B should remain 'non-prescribing'
  });

  it('has no effect when no break groups are currently active', () => {
    // Mock time to 14:00, all break groups end at 12:00
    // State should be unchanged
  });
});
```

10. Epic 8 — Non-Prescribing Pool

10.1 User Stories

US-NP-01

As a Resource Manager, I want to move a prescriber to the Non-Prescribing pool with a reason and optional note so that the team understands why they are unavailable.

Acceptance Criteria:

- [] Non-prescribing tile accepts drag-drop from the pool panel or any category tile
- [] On drop, a modal opens to select prescriber (if not already set) and reason
- [] Reasons: Admin, Training, Meeting, Lunch, Other
- [] Optional free-text note (e.g. "Team all-hands")
- [] "+" button on the non-prescribing tile opens the same modal without requiring a drag
- [] Prescriber is removed from any category tile allocation when moved to non-prescribing
- [] Prescriber status becomes `non-prescribing`

US-NP-02

As a Resource Manager, I want to return a prescriber from non-prescribing back to the available pool via the × button on their card.

Acceptance Criteria:

- [] Each prescriber card in the non-prescribing tile has a × button
- [] Clicking × sets the prescriber back to `online` status and removes their `NonPrescribingSlot`
- [] The prescriber immediately appears in the pool panel's "Available Pool" section

11. Reference Data & Configuration

11.1 Capacity Configuration (AHT per category)

Must be editable by admin users:

```
// Node.js API endpoint example
// GET /api/capacity-configs
// PUT /api/capacity-configs/:categoryId
app.put('/api/capacity-configs/:categoryId', async (req, res) => {
  const { categoryId } = req.params;
  const { orderAHTMins, messageAHTMins } = req.body;

  if (typeof orderAHTMins !== 'number' || orderAHTMins < 1 || orderAHTMins > 120) {
    return res.status(400).json({ error: 'orderAHTMins must be 1-120' });
  }
  if (typeof messageAHTMins !== 'number' || messageAHTMins < 1 || messageAHTMins > 60) {
    return res.status(400).json({ error: 'messageAHTMins must be 1-60' });
  }

  const updated = await db.capacityConfigs.upsert({ categoryId, orderAHTMins, messageAHTMins });
  res.json(updated);
});
```

11.2 Historical Volume Data (for BODCON projection)

Real historical order volumes by day-of-week, used for projected demand warnings:

```
# Stored in DB, configurable via admin UI
DAILY_ORDER_CONFIG = {
  "womens-health": [1968, 2150, 1767, 1609, 1471, 1537, 1685], # Sun-Sat
  "weight-management": [1801, 2035, 1886, 1890, 1896, 1967, 1535],
  "mens-health": [209, 311, 275, 259, 278, 263, 218],
  "sexual-health": [340, 420, 380, 350, 320, 380, 410],
  "dermatology": [311, 307, 318, 344, 343, 304, 287],
  "mental-health": [120, 180, 165, 155, 150, 160, 130],
  "general-health": [187, 280, 184, 185, 228, 194, 169],
}

def get_projected_remaining(category_id: str, day_of_week: int, hours_elapsed: float) -> int:
  """Returns expected remaining orders for the rest of today."""
  work_hours = 12.0
  expected_today = DAILY_ORDER_CONFIG[category_id][day_of_week]
  fraction_remaining = max(0, (work_hours - hours_elapsed) / work_hours)
  return round(expected_today * fraction_remaining)
```

11.3 Prescriber Data (initial roster)

The system is seeded with 55 prescribers across four roles. In production this should be sourced from the HR/Identity system:

- **Pharmacists (p-01 to p-20):** Women's Health, Men's Health, Dermatology, General Health, Sexual Health
- **Nurses (p-21 to p-40):** Full scope — all seven categories
- **GPs (p-41 to p-48):** All categories
- **Specialists (p-49 to p-55):** Restricted to specialist services (HRT, GLP-1, ADHD, HIV, Fertility)

12. Technical Architecture

12.1 Frontend

Layer	Technology
Framework	React 18 with TypeScript
Build tool	Vite
State management	React Context + <code>useReducer</code> (Redux-style, no external library)
Styling	Inline CSS-in-JS with CSS custom properties (design tokens)
Drag-and-drop	HTML5 native Drag and Drop API
Type safety	TypeScript strict mode

Design tokens (CSS custom properties):

```
--boots-blue: #05054B;      /* Primary brand colour */
--surface: #ffffff;
--surface-alt: #F8F9FB;
--border: #E2E6EF;
--fg1: #111827;
--fg3: #6B7280;
--space-4: 16px;
--r-lg: 10px;
--r-md: 6px;
--shadow-1: 0 1px 3px rgba(0,0,0,0.08);
```

12.2 State Architecture

All application state lives in a single `WorkforceContext` . State is mutated only through `dispatch(action)` calls:

```
// All action types
type Action =
  | { type: 'ALLOCATE_PRESCRIBER'; categoryId: string; prescriberId: string }
  | { type: 'DEALLOCATE_PRESCRIBER'; prescriberId: string }
  | { type: 'MOVE_PRESCRIBER'; prescriberId: string; fromCategoryId: string; toCategoryId: string }
  | { type: 'AUTO_ALLOCATE' }
  | { type: 'SET_PRESCRIBER_STATUS'; prescriberId: string; status: Prescriber['status'] }
  | { type: 'ADD_ORDERS'; orders: Order[] }
  | { type: 'ADD_NON_PRESCRIBING'; slot: NonPrescribingSlot }
  | { type: 'REMOVE_NON_PRESCRIBING'; prescriberId: string }
  | { type: 'ADD_APPOINTMENT'; appointment: Appointment }
  | { type: 'UPDATE_APPOINTMENT'; appointment: Appointment }
  | { type: 'DELETE_APPOINTMENT'; appointmentId: string }
  | { type: 'APPLY_BREAKS' }
  | { type: 'UPDATE_CAPACITY_CONFIG'; config: ServiceCapacityConfig }
  | { type: 'ADD_ACTIVITY_EVENTS'; events: PrescriberActivityEvent[] }
  | { type: 'LOG_ACTIVITY'; prescriberId: string; activityType: 'order' | 'message' }
  | { type: 'UPDATE_PERFORMANCE_CONFIG'; config: PerformanceMonitorConfig }
  // ...
```

12.3 Backend API (to be built)

The prototype runs client-side only. Production should expose a REST or WebSocket API for:

```
GET  /api/prescribers           → Prescriber[]
PUT  /api/prescribers/:id/status → Update status
GET  /api/orders?status=pending → Order[]
POST /api/allocations/auto      → Trigger auto-allocate, returns DayAllocation[]
PUT  /api/allocations/:categoryId → Update prescriber list for category
GET  /api/activity?prescriberId=... → PrescriberActivityEvent[]
POST /api/activity              → Log activity event
GET  /api/capacity-configs      → ServiceCapacityConfig[]
PUT  /api/capacity-configs/:categoryId → Update AHT config
GET  /api/appointments?date=YYYY-MM-DD → Appointment[]
POST /api/appointments          → Create appointment
PUT  /api/appointments/:id      → Update appointment
```

WebSocket events (for real-time updates):

```
{ "event": "order.created",    "data": { "order": Order } }
{ "event": "order.escalated",  "data": { "orderId": "...", "reason": "..." } }
{ "event": "prescriber.status", "data": { "prescriberId": "...", "status": "..." } }
{ "event": "activity.logged",   "data": { "event": PrescriberActivityEvent } }
```


12.4 Database Schema (suggested)

```
-- Core tables
CREATE TABLE prescribers (
  id VARCHAR PRIMARY KEY,
  name VARCHAR NOT NULL,
  initials VARCHAR(3) NOT NULL,
  role VARCHAR NOT NULL CHECK (role IN ('pharmacist','nurse','gp','specialist')),
  service_ids TEXT[] NOT NULL, -- Array of service IDs
  status VARCHAR NOT NULL DEFAULT 'offline',
  allocated_category_id VARCHAR REFERENCES service_categories(id)
);

CREATE TABLE orders (
  id VARCHAR PRIMARY KEY,
  service_id VARCHAR NOT NULL REFERENCES services(id),
  patient_ref VARCHAR NOT NULL,
  urgency VARCHAR NOT NULL CHECK (urgency IN ('routine','urgent','critical')),
  age_hours NUMERIC NOT NULL,
  value NUMERIC NOT NULL,
  prescriber_id VARCHAR REFERENCES prescribers(id),
  status VARCHAR NOT NULL DEFAULT 'pending',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  priority_score INTEGER DEFAULT 0
);

CREATE TABLE prescriber_activity_events (
  id VARCHAR PRIMARY KEY,
  prescriber_id VARCHAR NOT NULL REFERENCES prescribers(id),
  type VARCHAR NOT NULL CHECK (type IN ('order','message')),
  timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE appointments (
  id VARCHAR PRIMARY KEY,
  patient_ref VARCHAR NOT NULL,
  clinic_type_id VARCHAR NOT NULL REFERENCES clinic_types(id),
  prescriber_id VARCHAR NOT NULL REFERENCES prescribers(id),
  start_time TIME NOT NULL,
  duration_mins INTEGER NOT NULL DEFAULT 30,
  notes TEXT,
  status VARCHAR NOT NULL DEFAULT 'scheduled',
  appointment_date DATE NOT NULL DEFAULT CURRENT_DATE
);

-- Indexes for performance queries
CREATE INDEX idx_activity_prescriber_time
  ON prescriber_activity_events(prescriber_id, timestamp DESC);

CREATE INDEX idx_orders_status_service
  ON orders(status, service_id);
```

12.5 Deployment

- **Frontend:** Static site (React + Vite build) deployed to CDN / GitHub Pages
- **Base path:** `/bleaf-demo/` (configurable via Vite `base` setting)

- **Assets:** Content-hash filenames for cache busting — update `index.html` on each deploy

Appendix A: BODCON Escalation Actions

Level	Required Action	Comms Cadence
1	All non-essential PIMs cancelled. MD, RM, Medical Director and Ops Director to meet within 30 mins. Twice-daily standups until resolved.	Continuous updates
2	MD, RM, Medical Director and Ops Director notified to meet within 2 hours. Agree significant paid marketing and/or service suspensions.	Meeting within 2 hours
3	Consider easing marketing activity and/or implementing short firebreaks on services. Clinical ops provide regular Slack updates.	×3 daily updates
4	Clinical ops provide daily updates specifying corrective action until all guardrails turn green.	×2 Slack updates
5	Monitor as usual. 1 daily plan of action message.	×1 morning update

Appendix B: Prescriber Status Transitions

```
offline —————→ online (manual "Log in")
online —→ allocated (drag-drop or auto-allocate)
online —→ non-prescribing (drag to NP tile or modal)
online —→ scheduled (roster import)
allocated → online (deallocate / drag back to pool)
allocated → on-break (Apply Breaks button)
allocated → non-prescribing (drag to NP tile)
allocated → in-appointment (appointment starts)
on-break → online (manual re-allocation after break)
in-appointment → allocated (appointment ends, return to queue)
non-prescribing → online (× button on NP card)
```

Document generated from working prototype: <https://tommckay85-bit.github.io/bleaf-demo/>

Prototype branch: `claude/exciting-gauss-6kWCq`